

# NexRoute

## Next-Generation Autonomous Delivery System

(codename during development: AURA - Autonomous Utility & Robotic Assistant)

### Project Progress Summary & Roadmap

---

## 1. Project Idea (What NexRoute Is)

NexRoute is an autonomous indoor service robot for an office or lab environment. It picks up a small item (document, snacks, stationery, small package) from a sender location (X) and delivers it to a receiver location (Y), then returns directly to its base station. The robot is a general-purpose office service robot prototype, not just a document carrier. It carries the item on an open tray - there is no lock or servo-controlled compartment.

### Core building blocks:

| Component                             | Role                                                                 |
|---------------------------------------|----------------------------------------------------------------------|
| App (currently: ESP32's own web page) | Robot call, select sender/receiver, confirm pickup & delivery        |
| Database (not yet implemented)        | RFID UID, location, junction routes, delivery status                 |
| IR sensor array (8 channel)           | Follow the black line; will detect the base's wide stop marker       |
| RFID (RC522)                          | Identify physical locations: Junction (J1), Sender (X), Receiver (Y) |
| Ultrasonic (HC-SR04)                  | Stop safely if a person/object is in front; resume when clear        |

### End-to-end workflow (as designed):

- 1 Robot waits at base station (STANDBY).
- 2 App sends a call: pick sender X and receiver Y.
- 3 Robot follows the black line from Base towards X.
- 4 Near junction J1, an RFID key ring is scanned - robot recognizes J1 and turns according to the planned route.
- 5 Robot reaches X (RFID Card 1) and stops.
- 6 Sender places item on the open tray, confirms 'Item Loaded' on the App.
- 7 Robot does a 180-degree turn, follows the line back through J1 (turning again) towards Y.
- 8 Robot reaches Y (RFID Card 2) and stops.
- 9 Receiver takes the item, confirms 'Received' on the App.
- 10 Robot does a 180-degree turn and returns straight through J1 back to base.
- 11 Robot detects the base's wide black stop marker and goes back to STANDBY.

### State machine:

```
STANDBY → CALL_REQUESTED → GOING_TO_X → AT_X → ITEM_LOADED → GOING_TO_Y → AT_Y →  
DELIVERED → RETURNING_TO_BASE → STANDBY
```

Future vision: instead of a fixed X/Y pair, any office location (A, B, C, D, E, F, X, Y, Z ...) should be selectable as sender or receiver by whoever calls the robot from the App - the caller picks who the item goes to. The current build already has the interface for this (a dropdown of locations), but only the X→Y route is physically wired and working right now, since the full multi-junction track has not been built yet.

## 2. Hardware Setup (Current, Confirmed Working)

Controller: ESP32 Dev Module (30-pin variant - no GPIO 0, 16, or 17 broken out on header).

| Subsystem                              | Pin(s)                                                    | Notes                                                  |
|----------------------------------------|-----------------------------------------------------------|--------------------------------------------------------|
| IR sensor array (8ch, left&rarr;right) | 13, 15, 14, 27, 26, 25, 33, 32                            | Digital input, black-line-on-white logic               |
| Motor driver (L298N, no ENA/ENB)       | INA=22, INB=23, INC=18, IND=19                            | PWM via ledcWrite directly on IN pins                  |
| Ultrasonic (HC-SR04)                   | Trig=4, Echo=35                                           | Echo moved to input-only pin 35 to free GPIO4 for RFID |
| Buzzer                                 | GPIO 5                                                    | Non-blocking beep-beep pattern on obstacle             |
| RFID (RC522)                           | SCK=21, MOSI=12, MISO=34, SDA(SS)=2, RST=8, VCC=5V, GND=3 | RFID+5V (not GND)                                      |

Power: ESP32 is powered via USB (laptop/powerbank). L298N motor driver runs on a separate battery. RC522 and HC-SR04 run off the ESP32's own 3.3V/5V rails. Common ground between the ESP32 side and the motor battery side is required and already in place.

### Confirmed RFID tag UUIDs:

| Tag      | Represents  | UID (hex)   |
|----------|-------------|-------------|
| Key ring | Junction J1 | BC 74 C1 01 |
| Card 1   | Sender X    | AC 71 8F 02 |
| Card 2   | Receiver Y  | 09 B2 30 03 |

### 3. What Has Been Completed

#### Line following

- 8-channel IR sensor array read and converted into a weighted error value.
- PID control (Kp, Ki, Kd, tunable live from the web page) drives left/right motor speeds.
- Line-lost recovery: robot pivots toward the last known side until the line is found again.

#### Manual control

- Web-based joystick (Forward/Back/Left/Right/Stop) over the ESP32's own WiFi Access Point (LineBot\_ESP32 / 12345678, <http://192.168.4.1>).
- Keyboard arrow-key support in the browser.
- Fixed an early wiring/motor-direction bug where Left and Right were reversed; also fixed a damaged/loose L298N channel (IND / GPIO23) that was found and diagnosed with an isolated GPIO test sketch.

#### Ultrasonic obstacle detection

- HC-SR04 read non-blocking every ~60ms; obstacle threshold set to 15 cm.
- Robot stops automatically when an obstacle is closer than the threshold, in both LINE and MANUAL modes (Manual 'Backward' is still allowed even with an obstacle in front, so the robot can reverse away).
- Buzzer gives a 'beep-beep' pattern when an obstacle is detected, and repeats every 10 seconds if the obstacle is still there.

#### RFID (RC522) integration - hardware

- Extensive hardware debugging was needed: the module initially gave inconsistent/invalid version-register readings (0x00, 0xEE, 0x82) and failed the official self-test multiple times.
- Root causes found and fixed along the way: a broken breadboard power rail (3.3V was not reaching the module), loose/flaky jumper wires, and an earlier incorrect chip-select (SS) wiring scheme.
- Final working wiring: SCK=21, MOSI=12, MISO=34, SDA(SS)=2, RST tied directly to 3.3V.
- All 3 tags (J1 key ring, X card, Y card) now scan reliably and their UIDs were captured.

#### RFID integration - software

- UID matching logic added to recognize J1 / X / Y tags.
- checkRFID() drives the state machine: reaching X or Y stops the robot and waits for an App confirmation; scanning J1 triggers a junction turn.

#### State machine, junction turning, App-style control

- Full state machine implemented: STANDBY -> GOING\_TO\_X -> AT\_X -> GOING\_TO\_Y -> AT\_Y -> RETURNING\_TO\_BASE -> STANDBY.
- Junction (J1) auto-turn implemented as a timed, open-loop pivot (currently ~500ms, needs on-track tuning). Route confirmed: turn Left at J1 towards X, turn Left at J1 towards Y, go Straight through J1 when returning to base.

- 180-degree U-turn added at X (after 'Item Loaded') and at Y (after 'Received') before continuing (currently ~1000ms, needs on-track tuning).
- Base station detection (first pass): if all 8 IR sensors read black at the same time for ~150ms, this is treated as the wide stop marker and the robot goes to STANDBY. Not yet tested on a real marker.
- A 'DELIVERY' tab was added to the existing web page, acting as a stand-in App: Sender/Receiver dropdown (A-Z, but only X and Y are functional right now), 'Call Robot', 'Item Loaded', 'Received', and 'Cancel/Reset' buttons, plus a live status display of the current state.
- A bug where the Left/Right pivot direction was inverted (opposite convention from the tested manual-drive Left/Right) was found and fixed.

## 4. What Is Still Left To Do

### On-track calibration (immediate next step)

- Run the full Call -> X -> Item Loaded -> Y -> Received -> Base cycle on the real track.
- Tune JUNCTION\_TURN\_MS (currently 500ms) so the J1 turn lines the robot back onto the line correctly - not too little, not too much.
- Tune UTURN\_MS (currently 1000ms) the same way for the 180-degree turns at X and Y.
- Confirm the U-turn spin direction is sensible (currently hardcoded to one direction; flip if it turns the wrong way).
- Test and validate the base wide-marker detection with an actual wide black marker on the track.

### Multi-location (A-Z) support

- Physical track has not been designed yet beyond the single X/Y route through one junction (J1).
- Need to decide: can every location be reached through just one junction (branching design), or will multiple junctions (J1, J2, J3 ...) be needed?
- Once the physical layout is decided, each new location needs: an RFID tag with a captured UID, and a defined turn sequence (which junctions, which direction at each) to reach it.
- The code already has a generalized Sender/Receiver selector UI and a location list array as a starting point, but the actual routing table for anything beyond X/Y still needs to be written.

### Database

- No real database exists yet - RFID UIDs and routes are hardcoded directly into the ESP32 code.
- Per the original project spec, a database should store RFID UID -> location mapping, junction routes, and delivery status. This still needs to be designed and connected (e.g. via HTTP requests from the ESP32 to a backend, or a cloud database/service).

### A real standalone App

- Currently, the 'App' is simply the ESP32's own built-in web page (works fine as a functional stand-in, but is not a separate mobile/desktop application as described in the original project document).
- Decide whether a real app is required for the final submission, or whether the current web interface is an acceptable substitute - this affects how much extra work is needed here.

### Robustness / edge cases

- What should happen if the robot loses the line completely during an autonomous delivery run?
- What should happen if an RFID tag is missed/not scanned when expected (e.g. sensor read glitch)?
- Handling of multiple/overlapping call requests (right now only one delivery job at a time is assumed).
- General safety timeouts (e.g. if the robot is stuck waiting at AT\_X or AT\_Y for too long).

### Documentation / presentation

- Prepare a clear demo script for the lab presentation based on the final working flow.
- Take photos/short video of the working robot for the report/slides.

## 5. Quick Reference For Picking This Back Up

Main firmware file: **line\_follow\_manual\_ultrasonic\_v4.ino** (single ESP32 sketch containing line following, manual control, ultrasonic obstacle avoidance, RFID, and the state machine).

### Key tunable constants in the code:

| Constant                                        | Current value   | Purpose                                                    |
|-------------------------------------------------|-----------------|------------------------------------------------------------|
| OBSTACLE_DISTANCE_CM                            | 15.0            | Ultrasonic stop distance                                   |
| JUNCTION_TURN_MS                                | 500             | How long the J1 pivot turn lasts - needs on-track tuning   |
| UTURN_MS                                        | 1000            | How long the 180-degree turn lasts - needs on-track tuning |
| J1_TURN_TO_X / J1_TURN_TO_Y / J1_TURN_RETURNING |                 | Direction at J1 per leg of the trip                        |
| Kp, Ki, Kd                                      | 25.0, 0.0, 15.0 | Line-follow PID gains, adjustable live from the web page   |
| baseSpeed / manualSpeed                         | 150 / 180       | Motor speed for autonomous vs manual driving               |

### Web interface (LineBot\_ESP32 / 12345678, <http://192.168.4.1>):

- LINE FOLLOW tab - manual start/stop of plain line following, PID tuning sliders.
- MANUAL tab - joystick control.
- DELIVERY tab - Sender/Receiver dropdown, Call Robot, Item Loaded, Received, Cancel/Reset, and a live status line showing the current state and distance reading.

### Suggested next session's starting point:

Put the robot on the real track, open the DELIVERY tab, and run through the full Call → X → Item Loaded → Y → Received → Base cycle end to end, adjusting JUNCTION\_TURN\_MS and UTURN\_MS until the turns line up correctly. After that, decide on the physical layout for additional locations (A-Z) before extending the routing logic.

---

This document was generated from the full project conversation history to serve as a snapshot of progress and a reference for continuing work (by the project owner or by any AI assistant picking up the project later).